ORAL PROJECT PRESENTATION

IoT Anomaly Detection System

Stream Processing for Smart Manufacturing

Wind Turbine Factory Monitoring System

IU International University of Applied Sciences

Project - From Model to Production

Muhannad Al-Ahmad

Prof. Dr. Frank Passing

January 2026

01 Presentation Outline

02 - Project Goals
Problem definition & objectives

03 - Planning & Methodology
Approach and technology choices

04 - System Architecture
Technical design overview

05-06 - Data Pipeline
Ingestion, processing, storage

07-08 - ML Model
Isolation Forest implementation

09-10 - API & Dashboard
REST API + React UI demo

11-12 - Challenges & Results
Implementation insights

13-16 - Conclusion & Q&A
Summary and reflection

02 Project Goals & Objectives

The Problem

A wind turbine component factory needs to monitor production quality through sensor data. Shop floor employees have identified that **temperature, humidity, and sound levels** are key indicators of production anomalies and faulty items.

Key Requirements

- Process continuous sensor data streams
- Detect anomalies in real-time
- Provide standardized REST API
- Enable easy monitoring & maintenance

Project Objectives

1. Design System Architecture

Create scalable, maintainable data pipeline

2. Implement ML Model

Build anomaly detection using Isolation Forest

3. Build RESTful API

Expose model predictions as a service

4. Create Monitoring Dashboard

Real-time visualization for factory managers

03 Planning & Methodology

Development Approach

1. Research
2. Design
3. Implement
4. Test

Why Isolation Forest?

- ✓ **Unsupervised:** No labeled anomaly data required
- ✓ **Efficient:** Suitable for real-time scoring
- ✓ **Scalable:** Handles high-dimensional data
- ✓ **Interpretable:** Anomaly scores for transparency

Tech Stack

`Backend`
FastAPI (Python)

`Frontend`
React + Tailwind

`Database`
MongoDB

`ML Library`
scikit-learn

`Visualization`
Recharts

04 System Architecture

IoT Sensors
Temp, Humidity, Sound

→

FastAPI
REST Endpoints

→

ML Model
Isolation Forest

↓ ↓ ↓

MongoDB
Data Persistence

Dashboard
React Frontend

Alerts
Notifications

Scalable
Async processing, modular design

Maintainable
Clear separation of concerns

Adaptable
Model retraining capability

05 Data Pipeline: Ingestion

Sensor Data Model

```
class SensorReading:
  id: str
  temperature: float # 15-100°C
  humidity: float # 20-90%
  sound_level: float # 40-120dB
  timestamp: datetime
  sensor_id: str
```

Data Sources

1. **Synthetic Generator:** Gaussian distribution for normal, uniform for anomalies
2. **REST API:** POST /api/sensor/data for external sensors
3. **Simulation:** /api/sensor/simulate endpoint

Stream Processing Flow

1. **Receive Data**
   Async FastAPI endpoint receives JSON payload
2. **Validate**
   Pydantic models validate data types & ranges
3. **Store**
   Motor async driver writes to MongoDB
4. **Predict**
   Forward to ML model for scoring

06 Data Pipeline: Storage & Retrieval

MongoDB Collections

- **sensor_readings**
  Raw sensor data with timestamps

- **predictions**
  Anomaly scores, confidence, features

- **alerts**
  Severity levels, acknowledgment status

Why MongoDB?

- ✓ Flexible schema for evolving sensor data
- ✓ Excellent time-series performance
- ✓ Native JSON support

Data Retrieval APIs

```
# Get latest readings
GET /api/sensor/readings?limit=100

# Get predictions (anomalies only)
GET /api/predictions?anomalies_only=true

# Get active alerts
GET /api/alerts?unacknowledged_only=true
```

Statistics Endpoint

```
{
  "avg_temperature": 45.5
  "avg_humidity": 55.0
  "avg_sound_level": 70.0
  "anomaly_rate": 10.5
}
```

07 Machine Learning: Isolation Forest

How It Works

Isolation Forest isolates anomalies by randomly selecting features and split values. **Anomalies are isolated faster** because they have distinct values that separate them from normal points.

Anomaly Score Formula

$$s(x, n) = 2^{(-E(h(x))/c(n))}$$

Features Used

Temperature
15-100°C

Humidity
20-90%

Sound
40-120dB

Model Configuration

```
model = IsolationForest(
  n_estimators=100,
  contamination=0.1, # 10% anomalies
  max_samples='auto',
  random_state=42
)
```

Training Data

Normal Samples
1,000

Anomaly Samples
100

Distribution
Normal: Gaussian (temp μ=45, σ=8; hum μ=55, σ=10; sound μ=70, σ=10)
Anomaly: Uniform (temp 80-100°C; hum 10-25%; sound 100-120 dB)

08 Prediction Service

Prediction Endpoint

```
POST
/api/predict

# Request Body
{
  "temperature": 85.0,
  "humidity": 20.0,
  "sound_level": 105.0
}
```

```
# Response
{
  "anomaly_score": 0.85,
  "is_anomaly": true,
  "confidence": 0.92
}
```

Severity Classification

CRITICAL
Temp > 90°C OR Sound > 110dB OR Humidity < 15%

HIGH
Anomaly Score > 0.8

MEDIUM
Anomaly Score > 0.6

LOW
Anomaly Score ≤ 0.6

Model Retraining

`POST /api/model/retrain` allows updating the model with new production data for continuous improvement.

09 RESTful API Design

Sensor Data

`POST` `/api/sensor/data`
Submit sensor reading

`GET` `/api/sensor/readings`
Get historical data

`GET` `/api/sensor/latest`
Get latest reading

`POST` `/api/sensor/simulate`
Generate test data

Predictions

`POST` `/api/predict`
Get anomaly prediction

`GET` `/api/predictions`
Get prediction history

`POST` `/api/model/retrain`
Retrain ML model

Alerts & System

`GET` `/api/alerts`
Get active alerts

`PUT` `/api/alerts/{alert_id}/acknowledge`
Acknowledge alert

`GET` `/api/system/status`
System health

`GET` `/api/stats/summary`
Statistics

API Design Principles

🔄
Stateless
No server-side session

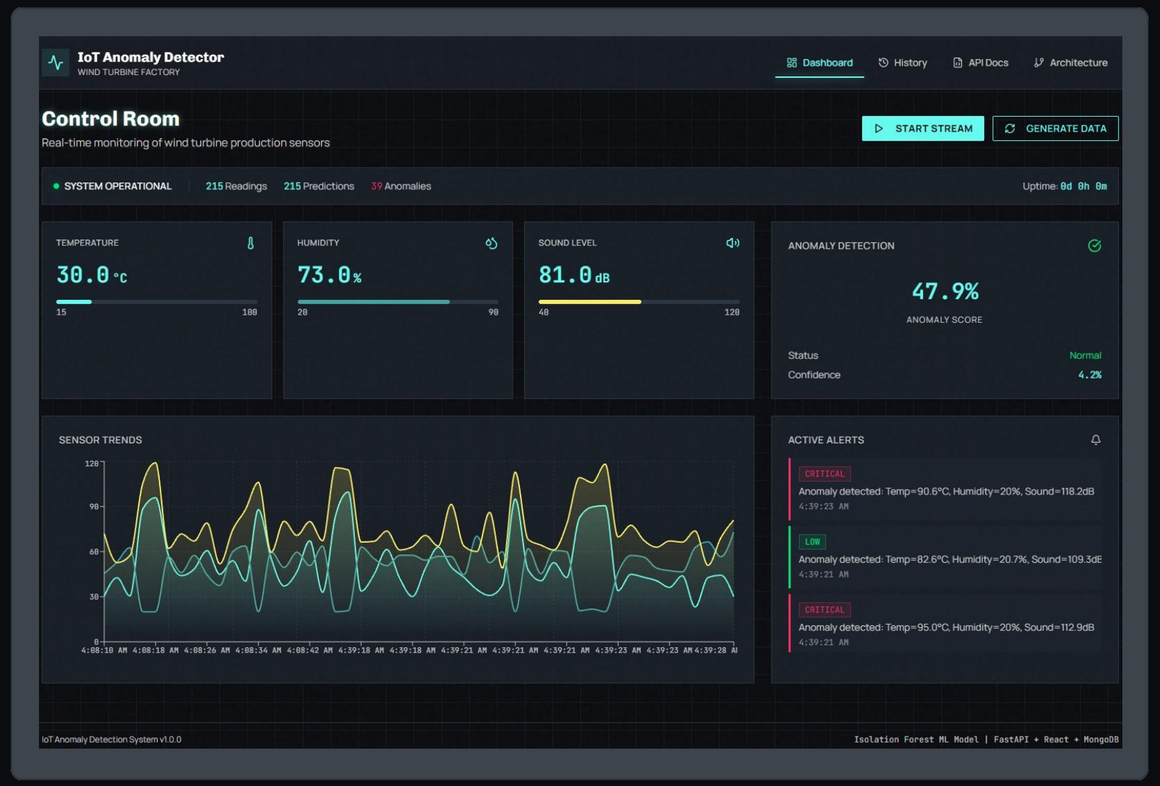
📦
JSON
Standard data format

✅
Validation
Pydantic models

📖
OpenAPI
Auto-generated docs

10 Dashboard Demonstration

Control Room Interface



1. Real-time sensor gauges with thresholds
2. Anomaly score display with status
3. Active alerts panel with severity

Key Features

📊 Sensor Trends Chart

Chart showing temperature, humidity, and sound over time with gradient fills

▶ Stream Simulation

"Start Stream" button generates continuous data every 2 seconds

🔔 Alert Management

Click to acknowledge alerts, severity-based color coding

📈 System Status

Total readings, predictions, anomalies, and uptime display

11 Challenges & Solutions

Challenge: Model Integration

Integrating ML model with async web framework while maintaining low latency

SOLUTION
Pre-loaded model on startup with joblib serialization, synchronous prediction calls wrapped in async endpoints

Challenge: Real-time Updates

Dashboard needs live data without WebSocket complexity

SOLUTION
Polling with 5-second interval, optimized MongoDB queries with proper indexing

Challenge: Data Simulation

No real sensor data available for development

SOLUTION
Synthetic data generator with configurable anomaly rate (20%), realistic Gaussian distributions

Challenge: Model Accuracy

Ensuring model generalizes well to unseen patterns

SOLUTION
Contamination parameter tuned to 10%, model retraining endpoint for continuous improvement

12 Results & Achievements

16	100%	4	<50ms
API Endpoints	Test Success Rate	Dashboard Pages	Prediction Latency

Goals Achieved

- ✓ Continuous stream processing via REST API
- ✓ Real-time anomaly detection with Isolation Forest
- ✓ Standardized RESTful API with OpenAPI docs
- ✓ Interactive monitoring dashboard
- ✓ Alert system with severity classification
- ✓ Model retraining capability

System Qualities

Scalability
Async processing, modular architecture

Maintainability
Clear code structure, documentation

Adaptability
Model retraining, configurable thresholds

13 Reflection & Lessons Learned

Key Learnings

Model as a Service

Packaging ML models as REST APIs requires careful consideration of latency, serialization, and error handling. Pre-loading models significantly improves response time.

Data Pipeline Design

Async processing with MongoDB Motor driver handles concurrent requests efficiently. Proper data validation at ingestion prevents downstream issues.

User Experience

Real-time dashboards need visual hierarchy and clear status indicators. Color-coded severity helps operators quickly identify critical issues.

What I Would Do Differently

→ **WebSocket Integration**
Replace polling with WebSocket for true real-time updates

→ **Time-Series Database**
Consider InfluxDB or TimescaleDB for better time-series performance

→ **Model Versioning**
Implement MLflow for model tracking and A/B testing

Constraints Encountered

- No real sensor hardware for true stream testing
- Limited historical data for model validation
- Single-node deployment (not distributed)

14 Conclusion

Summary

Successfully designed and implemented an **IoT Anomaly Detection System** that processes sensor data streams, applies machine learning for anomaly detection, and provides a comprehensive monitoring dashboard for factory operations.

Answering Key Questions

Challenges of ML Integration?
Model serialization, latency optimization, error handling for edge cases

Constraints of ML as a Service?
Memory usage, prediction latency, model versioning, retraining triggers

Data Requirements Met?
Async ingestion, MongoDB persistence, efficient retrieval with proper projections

Future Improvements

WebSocket Streaming
True real-time updates without polling

Authentication
Role-based access for operators and managers

Notifications
Email/SMS alerts for critical anomalies

Cloud Deployment
Kubernetes orchestration for scalability

Advanced ML
LSTM for time-series, ensemble methods

15 References

Bibliography

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). *Isolation Forest*. Eighth IEEE International Conference on Data Mining.

FastAPI Documentation. (2024). *FastAPI - Modern, Fast Web Framework for Building APIs*. https://fastapi.tiangolo.com/

Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.

MongoDB Documentation. (2024). *The MongoDB Database*. https://www.mongodb.com/docs/

React Documentation. (2024). *React - A JavaScript Library for Building User Interfaces*. https://react.dev/

Thank You

Questions & Discussion

GitHub Repository

`https://github.com/Muhannad-Alahmad/IoT-Anomaly-Detector`

Dashboard (local): http://localhost:3000

API Docs (local): http://127.0.0.1:8001/docs